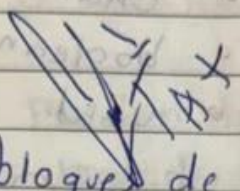
UNIDAD 2

Modelo de Programación funcional:

El Modelo de la Programación funcional es un Paradigma declarativo que basa el desarrollo de Software en el uso de funciones matemáticas Puras y datos inmutables, en lugar de cambiar estados o modificar Variables através del tiempo.



Conceptos clave:

- **Funciones Puras:** Son bloques de código que, con los mismo Valores de entrada, siempre devuelven exactamente el mismo resultado y no generan efectos Secundarios en el Sistema.
- **Inmutabilidad:** Los datos no se Pueden modificar una Vez Creados. Si necesitas Cambiar Un Valor se genera Uno nuevo basado en el anterior.
- **Transparencia referencial:** una expresión Puede ser reemplazada Por su Valor resultante Sin alterar el comportamiento general de Programa.
- **Composición de funciones:** Consiste en Combinar dos o más funciones Para Crear Una nueva operación más Compleja de manera Secuencial.

Ventajas Principales:

- **Código Predecible:** Al no existir estados compartidos ni mutaciones ocultas, los errores (bugs) disminuyen de forma notable.
- **Facilidad de Pruebas (Testing):** Las funciones Puras Son muy fáciles de Probar de forma aislada Porque no depende de contextos externos.

Concurrencia Segura. Como los datos no cambian, múltiples procesos o hilos pueden leer la misma información al mismo tiempo sin riesgo de corromper los datos.

La Programación funcional no es más que un Paradigma de Programación, es decir, es una forma en la cual podemos resolver diferentes problemáticas.

Cuando nos encontramos desarrollamos Software utilizando este Paradigma estaremos trabajando principalmente con funciones evitaremos los datos mutables, así como el hecho de compartir estados entre funciones.

Con este Paradigma las funciones serán tratadas como ciudadanos de primera clase.
Las funciones podrán ser asignadas a variables además podrían ser utilizadas como entrada y salida de otras funciones.

Con este Paradigma las funciones serán tratadas como ciudadanos de primera clase.
Las funciones podrán ser asignadas a variables además podrían ser utilizadas como entrada y salida de otras funciones.
A Las funciones que pueden tomar funciones como parámetros y devolver funciones como resultado serán conocidas como función de orden superior.

La Programación funcional es un Paradigma declarativo nos enfocaremos en "qué" estamos haciendo y no en "como" se está haciendo que sería el enfoque imperativo. Esto quiere decir que nosotros expresamos nuestra lógica sin describir controles de flujo.

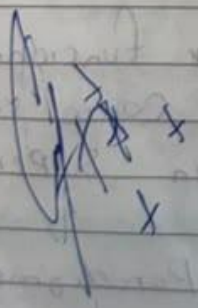
Calculo lambda de Alonso Church

El Cálculo lambda, desarrollado por Alonzo Church, es un sistema matematico que sirve para representar funciones y operaciones mediante expresiones llamadas términos lambda. Es una de las bases teóricas de la Programación funcional.

Su estructura básica es: $\lambda x.M$

Donde:
(λ) → Indica que estamos definiendo una función.
(x) → es la variable o el parametro.
(M) → es el resultado o expresión de la función.

- El λ-Cálculo fue inventado por Alonso Church en la década de 1930.
- Originalmente fue inventado como parte de un sistema formal para modelar la matematica.
- Es utilizado para estudiar la computabilidad.
- En los 1960s, Peter Landin muestra que se puede usar para dar Semántica a los lenguajes de Programación (imperativos)



Elixir y Erlang

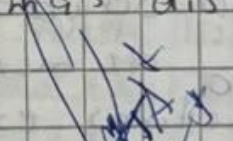
Son dos lenguajes de Programación funcionales que comparten la misma máquina Virtual, conocida como (BEAM), y el mismo ecosistema subyacente.

Principales diferentes?

- Sintaxis y Origen: Erlang nació a finales de los 80 en Ericsson para sistemas de telecomunicaciones y usa una sintaxis Parecida a Prolog.

Elixir se creó en 2011 Para ofrecer una sintaxis moderna y amigable inspirada en Ruby.

- Enfoque de uso: Erlang: Se diseño con un enfoque de abajo hacia arriba Para sistemas distribuidos complejos.



Elixir busca facilitar el desarrollo web y general con herramientas integradas como el framework

- Curva de aprendizaje: Elixir resulta más accesible Para nuevos desarrollos gracias a su documento de alta calidad y herramientas Modernas.

Qué tienen en común:

- La Maquina Virtual (BEAM): Ambos compilan a un código intermedio que corre en la misma maquina virtual lo que les otorga una concurrencia masiva y alta disponibilidad.

Interoperabilidad: El código en Exir Puede llama,

a funciones de Erlang sin problemas de rendimiento ni traducciones complejas.

Tolerancia a fallos: Los dos usan el modelo de actores y la filosofía de "dejar que falle" (let it crash) para construir sistemas resilientes.

WhatsApp utiliza principalmente Erlang en sus servidores para manejar la mensajería y la concurrencia a escala masiva.

Gestión de conexiones y chat: La gran mayoría del código del servidor de WhatsApp encargado de enrutamiento de mensajes y mantenimiento de conexiones activas está escrito en Erlang.

Alto rendimiento y concurrencia: Permite mantener millones de conexiones simultáneas abiertas en un sólo servidor, gracias a los procesos ligeros de la máquina virtual BEAM.

Tolerancia a fallos: si un proceso de usuario falla, este se reinicia de manera aislada sin afectar el resto del sistema garantizando una alta posibilidad.

Actualizaciones en caliente: Permite actualizar el código del servidor en tiempo real sin necesidad de apagar el servicio ni interrumpir a los usuarios.

Motor personalizado: (swartz) WhatsApp utiliza una versión modificada y optimizada.

Tarea:

D | M | A

Scribe

Investigar Funciones de orden superior (map, filter, fold/reduce) sobre listas.

Son aquellas que pueden recibir otras funciones como argumentos retornar funciones como resultados, o ambas cosas.

En la Programación funcional, map, filter y fold (también conocida como reduce) son los tres pilares fundamentales para procesar colecciones de datos (como listas o arreglos) de forma limpia y declarativa, evitando bucles manuales (como for o while).

1. Función map (Mapear / Transformar)

La función map toma una función y una lista como argumentos. Aplica esa función a cada elemento de la lista de forma individual y devuelve una nueva lista con los resultados transformados, manteniendo exactamente el mismo tamaño que la lista original.

- Propósito: Transformar los datos de una colección.

2. Función filter (filtrar / seleccionar)

La función filter toma una función de condición (un predicado que evalúa a verdadero o falso true o false) y una lista. Recorre la lista evaluando cada elemento y construye una nueva lista que contiene únicamente aquellos elementos que cumplen con la condición.

- Propósito: Seleccionar un subconjunto de datos basándose en criterios específicos.

3. Función fold / reduce (Reducir / Acumular).

La función fold comúnmente llamada reduce en lenguajes como Python, JavaScript o Ruby) toma una función acumuladora, un valor inicial (opcional o obligatorio según el lenguaje) y una lista. Su objetivo es procesar los elementos de la lista uno por uno combinalos con un acumulador para reducir toda la estructura a un único valor final.

- Proposito: Consolidar una lista entera en un solo resultado como una suma total, un producto, encontrar el valor máximo o concatenar textos.

Función	Qué hace	Tamaño del Resultado	Tipo de retorno
MAP	Transforma cada elemento aplicado una función	Igual a la lista Original	Nueva lista.
filter	Selecciona elementos basándose en una condición lógica.	Menor o igual a la lista Original	Nueva lista filtrada.
fold / reduce	Combina y acumula todos los elementos paso a paso	Un único Valor	Valor Primitivo o objeto Consolidado.

NOTACIÓN

Prefijas, Infijas y Posfijas

Aspectos	Prefija	Infija	Posfija
Definición	Es una forma de escribir expresiones en la que el operador se coloca antes de los operadores sobre los que actúa	Es una forma de escribir expresiones en la que el operador se coloca entre operadores. Es la notación utilizada habitualmente en matemáticas	Es una forma de escribir expresiones en la que el operador se coloca después de los operadores. También se conoce como notación polaca inversa.
Características Principales	Los operadores aparecen al inicio de cada operación y permiten representar expresiones sin utilizar paréntesis cuando la expresión está correctamente.	Los operadores aparecen entre los valores o variables que intervienen en la operación. su interpretación depende de las reglas de precedencia, asociatividad y uso.	Los operadores aparecen después de los valores o variables que utilizan. La expresión puede interpretarse sin paréntesis mediante la posición de su elemento
Posición del operador	Antes de los operadores	Entre los operadores	Después de los operandos.
Posición de los operandos	Después del operador	A ambos lados del operador	Antes del operador.
Forma general	operador, operando, operando	operando, operador operando.	operando, operando operador.
Ejemplo de suma	+ A B	A + B	AB +
Ejemplo de resta	- A B	A - B	AB -
Ejemplo de multiplicación	* A B	A * B	A B *
Ejemplo de división	/ AB	A / B	AB /
Uso de Paréntesis	No requiere paréntesis para expresar el orden en una notación prefija válida	Puede necesitar paréntesis para modificar o aclarar el orden de las operaciones	No requiere paréntesis para expresar el orden en una notación posfija valida.
Prioridad de Operadores	La estructura de la expresión determina que operación se realiza primero	Se aplican reglas de prioridad como realizar multiplicaciones antes que sumas.	La posición de los operadores determina el orden de evaluación.
Uso de Pilas	Evita ambiguedades relacionadas con la prioridad. Las pilas permiten almacenar operadores y calcula los resultados de las operaciones	Las pilas pueden utilizar para convertirse expresiones o analizar el orden de las operaciones	Las pilas permiten almacenar operandos y efectar operaciones conforme aparecen los operadores.

Intervalos, Operadores, y Operación Perezosa en haskell.

En Haskell, el manejo de colecciones infinitas y el Procesamiento eficiente de datos se fundamentan en su Paradigma funcional. El concepto Central que one a los Intervalos, los operadores y la evaluación Perezosa es la Postergación del cálculo hasta que es estrictamente necesario

1. Intervalos (Ranges):

Los Intervalos en Haskell son una Sintaxis comóda (azúcar sintáctico). Para generar listas de elementos ordenados. utilizan la notación de corchetes [...] y se basan Internamente en la clase de tipos Enum.

Intervalo básico: Genera una secuencia consecutiva.

```haskell
[1..5] -- Devuelve: [1,2,3,4,5]
['a'..'e'] -- Devuelve ['a','b','c','d','e']
```

Intervalo con Paso (step): Permite especificar la diferencia.

2. Operación Perezosa (Lazy Evaluation).

Haskell utiliza Por defecto la evaluación Perezosa (o llamada Por necesidad). Esto significa que las expresiones no se evalúan cuándo se define, sino sólo cuándo su resultado es indispensable Para otra Computación (Por ejemplo, Para mostrarlo en Pantalla.

Mecanismo Interno: Los Thunks.

Cuándo Creas una lista como [1..] Haskell no

Llena la memoria de la computadora con infinitos numeros. En su lugar, crea un thunk (un puntero o "promesa" de calculo. Sólo cuándo pides ver o usar una parte de la lista, el thunk se evalúa lo minimo necesario.

Ventajas Prácticas:
- Estructuras de datos infinitas: puedes definir flujos de datos sin fin y consumir solo lo que necesitas.

haskell:
-- Toma los primeros 5 elementos de una lista infinita:
take 5 [1..] -- [1,2,3,4,5]

Operación Perezosa (Lazy Evaluation)
Haskell utiliza por defecto la evaluación perezosa (o llamada por necesidad). Esto significa que las expresiones no se evalúan cuándo se definen, sino solo cuando su resultado es indispensable para otra computación. (Por ejemplo: Para mostrarlo en pantalla)

Mecanismo Interno: Los Thunks.
Cuándo creas una lista como [1..] Haskell no llena la memoria de la computadora con infinitos números. En su lugar, crea un thunk (un puntero o "promesa de calcula. Solo cuándo pider ver o usar una parte de la lista, el thunk se evalúa lo minimo necesario.